

The Complexity Ledger

SUBThe Complexity Ledger

p-rick research program

p-rick — an independent research program on missing software: An accounting discipline for the one quantity design moves most.

The Complexity Ledger

SUBThe Complexity Ledger

p-rick research program

Working Paper P-012 · v1.0 · October 7, 2026 · CC BY 4.0

Abstract — Every discipline that moves matter has an accounting discipline — thermodynamics tracks energy, double-entry bookkeeping tracks money, industrial engineering tracks waste — and the discipline that moves *complexity* has folklore instead. Larry Tesler stated the law in the 1970s while at Xerox PARC: every application has an inherent amount of irreducible complexity, and the only design question is who deals with it. Fifty years later the law is quoted in design talks, contradicted by every “we simplified it!” launch announcement, and measured by no one. The consequences are not aesthetic. Teams ship simplifications that relocate complexity to two million users in unmeasured quantities and then measure the engorgement as “engagement” or “support tickets” without ever recording the transaction; regulators mandate “simpler” interfaces that relocate complexity into terms of service; platforms relocate complexity into developers, developers into workarounds, workarounds into the brittleness that Hyrum’s Law then makes load-bearing. This paper formalizes the **conservation of complexity** as a design theory — distinguishing *reduction* of accidental complexity (Brooks) from *relocation* across system boundaries (Ashby’s requisite variety as the conservation constraint) — and specifies the missing accounting instrument: the **complexity ledger**, a per-decision record of how much complexity moved, from whom, to whom, in what measurable proxies, with whose consent. The ledger’s units are deliberately operational rather than ontological: five sinks (code, interaction, operation, documentation, deferral), each with field-measurable proxies already collected by engineering organizations and never joined — static complexity deltas, added interaction steps, runbook and alert load, documentation volume, support-ticket migration, onboarding drop-off. We specify the record format, the budget reconciliation at release cadence, the consent principle (relocating complexity to users without disclosure is a design externality, the software equivalent of dumping), and the falsifiable predictions the theory makes (waterbed signatures in compiler flag growth, ticket-migration signatures after UI “simplifications”, documentation-volume compensation after API redesigns). Every component measurement exists in production somewhere; the composition — an accounting discipline for complexity — exists nowhere, in no company, no standard, no curriculum. The gap is STRONG, the theory is falsifiable, the instruments are cheap, and the payoff is the first honest answer to the question every steering committee actually asks — *what did we simplify, and who is paying for it now?*

Keywords: p-rick

p-rick working paper P-012 · series IV (assumptions) · draft 1.0

Every discipline that moves matter has an accounting discipline — thermodynamics tracks energy, double-entry bookkeeping tracks money, industrial engineering tracks waste — and the discipline that moves *complexity* has folklore instead. Larry Tesler stated the law in the 1970s while at Xerox PARC: every application has an inherent amount of irreducible complexity, and the only design question is who deals with it. Fifty years later the law is quoted in design talks, contradicted by every “we simplified it!” launch announcement, and measured by no one. The consequences are not aesthetic. Teams ship simplifications that relocate complexity to two million users in unmeasured quantities and then measure the engorgement as “engagement” or “support tickets” without ever recording the transaction; regulators mandate “simpler” interfaces that relocate complexity into terms of service; platforms relocate complexity into developers, developers into workarounds, workarounds into the brittleness that Hyrum’s Law then makes load-bearing. This paper formalizes the **conservation of complexity** as a design theory — distinguishing *reduction* of accidental complexity (Brooks) from *relocation* across system boundaries (Ashby’s requisite variety as the conservation constraint) — and specifies the missing

accounting instrument: the **complexity ledger**, a per-decision record of how much complexity moved, from whom, to whom, in what measurable proxies, with whose consent. The ledger’s units are deliberately operational rather than ontological: five sinks (code, interaction, operation, documentation, deferral), each with field-measurable proxies already collected by engineering organizations and never joined — static complexity deltas, added interaction steps, runbook and alert load, documentation volume, support-ticket migration, onboarding drop-off. We specify the record format, the budget reconciliation at release cadence, the consent principle (relocating complexity to users without disclosure is a design externality, the software equivalent of dumping), and the falsifiable predictions the theory makes (waterbed signatures in compiler flag growth, ticket-migration signatures after UI “simplifications”, documentation-volume compensation after API redesigns). Every component measurement exists in production somewhere; the composition — an accounting discipline for complexity — exists nowhere, in no company, no standard, no curriculum. The gap is STRONG, the theory is falsifiable, the instruments are cheap, and the payoff is the first honest answer to the question every steering committee actually asks — *what did we simplify, and who is paying for it now?*

1 Introduction

In the middle 1970s, at Xerox PARC, a researcher named Larry Tesler grew tired of watching designers move difficulty around and call the result simplicity. He wrote down what he called the law of conservation of complexity: every application must have an inherent amount of irreducible complexity; the only question is where that complexity lives. Tesler’s own gloss is the part everyone forgets: *“the user or the designer must deal with it.”* Not “complexity is bad” — complexity *placement* is the design act. Tesler spent a career acting on the law (modeless editing, cut and paste, the dynamic debugging he built into the Smalltalk browser) and died in 2020 with his law quoted in a thousand conference talks and operationalized in exactly zero engineering processes.

The law’s neglect has a cost, and the cost is now macroscopic. The software industry’s characteristic failure of the last decade is not bad engineering; it is unaccounted complexity relocation. Consider the pattern, which recurs at every scale:

- A platform “simplifies” its API by hiding configuration behind auto-detection. The complexity moves from the integration engineer’s configuration file to every user’s failure modes, which now occur inside a black box at runtime, in production, at 3 a.m.
- A product “simplifies” its settings by removing them. The complexity moves into forums, where users reconstruct the removed decision tree as folklore, and into power users’ home-grown tooling, which Hyrum’s Law then makes load-bearing.
- A regulation mandates a “simpler” consent flow. The complexity moves into the dark-pattern engineering of the pre-consent screens, which the regulation does not see because the accounting does not exist.
- A team migrates to a framework that “removes boilerplate.” The complexity moves into the framework’s upgrade path, surfaced years later as a breaking-change migration measured in engineer-months.

Each of these transactions is a real economic event: complexity — which is to say, human attention, error rate, and time — moved from one party to another. None of them is recorded anywhere. No commit message says “relocated 3 complexity units to 2M users.” No launch review compares where the difficulty went against where it came from. The industry that instrumented request latency to the microsecond has no instrument for the one quantity it moves around most.

The thesis in one sentence: **complexity is conserved across design boundaries — reduction of accidental complexity is possible but rare and hard, relocation is the**

common case — and the discipline that makes relocation visible, consensual, and budgeted is an accounting specification, not a new methodology.

This is a theory paper with an instrument. Section 2 states the theory precisely enough to be falsifiable. Section 3 surveys the landscape — folklore, measurement fragments, adjacent disciplines — and verifies the gap. Section 4 explains why the gap survived. Section 5 specifies the ledger. Section 6 gives the evaluation design, including the natural experiments that could kill the theory. Sections 7–9 do what the program requires: what this is not, objections confronted, limitations.

2 The theory, stated precisely

2.1 Definitions

Complexity, for the purposes of a conservation law, cannot be a single ontological quantity — Section 8 confronts this at length — so the theory is stated over an operational definition: the complexity of a system-region is the *capacity-adjusted cost of achieving correct outcomes within that region*. “Cost” decomposes into the proxies any accounting can use: time, error rate, cognitive load, and maintenance labor. “Capacity-adjusted” is the crucial qualifier: one hour of difficulty for an expert is not one hour of difficulty for a civilian; a relocation is *transductive* when it moves capacity-adjusted cost upward, from experts to non-experts.

A **boundary** is any design-relevant division between system regions maintained by different parties: the API surface, the UI, the documentation, the deployment pipeline, the support organization, the user’s habits. The choice of boundaries is itself a design decision — an accounting discipline makes boundary choice explicit rather than inherited.

Reduction is a design change that lowers total capacity-adjusted cost across the whole system with no compensating increase elsewhere. Brooks’s distinction is the theoretical warrant: essential complexity (inherent in the problem) cannot be reduced; accidental complexity (artifacts of the chosen solution) can. Brooks’s “No Silver Bullet” claim is precisely that accidental-complexity reduction yields *linear* gains while the demand is for an *order-of-magnitude* gain — a prediction forty years accurate and counting.

Relocation is a design change that lowers capacity-adjusted cost in one region and raises it in another. Ashby’s law of requisite variety is the conservation constraint: a controller must have at least as much variety as the disturbances it must regulate. Delete variety from the controller (the software), and the variety demand does not vanish — it relocates to whatever remains in the loop: the user, the operator, the documentation, or the failure rate. Ashby, 1956, is the formal statement of what Tesler said informally and what designers experience as the waterbed: push it down here, it surfaces there.

2.2 The law and its falsifiable form

Conservation claim (weak form). For design changes that alter interfaces and not underlying solution structure — the overwhelming majority of what product engineering does — total capacity-adjusted cost across all regions is approximately conserved; measured “simplifications” are predominantly relocations.

Conservation claim (strong form). The approximation error is structured, not random: relocations systematically *understate* total cost when moving complexity to lower-capacity regions (the transduction penalty — a cost is amplified when received by a party with less capacity to bear it), so naive conservation is a *lower bound* on the true post-change cost.

The weak form is directly falsifiable: instrument the five sinks (§5.2) before and after a sequence of interface redesigns; if total measured cost systematically *drops* with no compensating

rise, relocation theory is wrong and something like genuine mass-simplification exists at product cadence. The strong form adds a testable asymmetry: relocations toward users should show total-cost *inflation* (the same nominal difficulty costs more downstream), which the ledger can detect as negative-sum transactions. Note what the theory does *not* claim: it does not claim all complexity is irreducible (that would make it unfalsifiable orthodoxy); it claims reduction is rare, hard, and trackable, and that the common case masquerading as reduction is relocation.

2.3 The five sinks

Relocations have five observable destinations, which together exhaust the space in a way that makes accounting closed:

1. **Code** — internal complexity of the artifact: static measures, dependency mass, flag count, migration surface.
2. **Interaction** — the user’s side of the boundary: steps, decisions, state the user must hold, error recovery they must perform.
3. **Operation** — the running side: runbooks, alert load, toil, incident frequency.
4. **Documentation** — the corpus that externalizes cognition: docs, tutorials, forum answers, tribal knowledge.
5. **Deferral** — time-shifted complexity: technical debt, workarounds, deprecated-but-alive paths. Deferral is the sink with interest.

A closed accounting over five sinks is what makes the ledger’s double-entry possible: any “reduction” claim must show which sink absorbed the relocated cost, and a claim that no sink absorbed it is a claim of genuine reduction — which the theory says should be rare and which the ledger will therefore *surface as the exceptional, headline event it is*.

3 The landscape: folklore, fragments, and the empty composition

The gap verification follows the program’s method: everything adjacent exists; the accounting discipline does not.

The folklore exists and is load-bearing. Tesler’s law (1970s, PARC) is canonical design culture. Brooks’s essential/accidental distinction (1986) is canonical software culture. The waterbed effect is canonical compiler culture — optimization folklore holding that suppressing complexity in one stage forces it to resurface in another; compilers are also the field’s best natural laboratory, having visibly converted “simpler source languages” into “flag-space explosion” (the classic observation that the compiler driver’s hundreds of flags are where the abstraction’s unresolved decisions live). Spolsky’s law of leaky abstractions (2002) states the failure mode of complexity relocated *downward* into a leak-proof-looking layer. Hyrum’s Law — users will depend on any observable behavior, documented or not — states why relocated complexity becomes contractual: the workaround built to absorb relocated complexity is now a dependency. Cunningham’s technical-debt metaphor (1992) is the deferral sink named. The folklore is complete. It is also completely unoperationalized: no one has ever stated what to *measure*, where to record it, or what reconciliation would look like.

The measurement fragments exist and are disconnected. Static complexity measurement is fifty years old (McCabe’s cyclomatic complexity, 1976; the maintainability-index family; the language workbench’s complexity smells), and every large codebase has dashboards for it. Interaction cost is measured by the HCI tradition: keystroke-level modeling (Card, Moran, Newell, 1980), time-on-task, error rate, SUS — an entire apparatus that lives in usability labs, never in

release engineering. Operational load is measured by the SRE tradition: alert volume, toil surveys (the SRE book’s own toil accounting), incident frequency. Documentation cost is countable (corpus size, read time, search success rate) and almost never counted. User-side absorption is measured by the organization’s most honest instrument — support tickets — and reported as a *support problem* rather than a complexity-relocation receipt. Cognitive-dimensions-of-notations research (Green, 1989; Green and Petre, 1996) built the vocabulary for notational complexity and never built the accounting. Each sink has a mature measurement community; no one has joined the five series into one ledger, because no one owns the transaction.

Adjacent disciplines solved the accounting pattern and did not import it. Double-entry bookkeeping is the oldest and best precedent: every transaction has two legs, forcing the question “from whom, to whom.” Thermodynamics is the conservation precedent: energy accounting works over operational proxies, not ontology. Environmental regulation is the externality precedent: impact assessments exist precisely because relocations with diffuse costs and concentrated benefits are invisible to the transacting parties. Industrial engineering’s waste accounting is the process precedent. The pattern is not novel; its application to software design transactions is.

Practitioner culture gestures at the ledger without building it. Design-decision records (ADRs, DDRs) capture *rationale* but not *magnitude* or *destination*. Complexity budget reviews exist in informal form at strong engineering cultures — the code review that pushes back “this moves the burden to on-call” is a verbal ledger entry, made once, unrecorded, unmeasured. Steering committees ask “what did we simplify?” and receive adjectives.

VERDICT: STRONG. No incumbent instrument exists for complexity accounting; every measurement primitive exists; the folklore is universally believed and never operationalized; the demand (steering committees, regulators, platform/developer relations) is expressed constantly in unanswerable form. A gap that is folklore-complete and instrumentation-empty is the cleanest gap class this program has specified since P-005.

4 Why the gap survived

Complexity moves are excellent strategy when unpriced. Relocating complexity to users converts engineering cost into diffuse user time — the transaction is profitable precisely because the ledger does not exist. Every “smart defaults that occasionally need overrides” design, every “zero-config until it breaks” deployment story, every consumer product whose manual is a forum thread is a profitable unrecorded relocation. The parties that profit built the tooling; the tooling does not record their transactions.

The quantity has no unit, so no one owed anyone a number. Energy had no unit until Joule; software complexity has resisted its unit because the ontological question (what IS complexity?) ate the operational question (what does it cost, where?). The measurement communities grew up in separate buildings — usability, SRE, static analysis, support — with separate vocabularies, so the transaction never had a place to be joined even in principle. Accounting needs a closed set of accounts more than it needs a true ontology of the underlying quantity; GDP is not a natural kind either.

Simplification is the industry’s marketing language. A discipline that shows most “simplification” to be relocation is a discipline that contradicts launch announcements, performance reviews, and conference keynotes. The folklore survives because it is said in the abstract and never applied to the specific — Tesler’s law quoted in the morning, “our new streamlined experience” shipped in the afternoon.

Reduction DOES happen, and its possibility inoculates the rest. Genuine accidental-complexity reduction is real (garbage collection over manual memory; packet switching over circuit engineering; the gradual disappearance of build-path hell in modern

package managers). Because reduction is occasionally achieved, every relocation can claim to be one. The ledger is precisely the instrument that separates the claims.

5 The specification

5.1 The relocation record

Every design decision that alters an interface between sinks emits a **relocation record** — the ledger’s fundamental transaction, structured as double-entry:

- **Motivating change.** What was changed, in the vocabulary of the design system (API surface, settings, flow, pipeline stage).
- **Claimed direction.** What the change claims about complexity (the launch-announcement form: “simpler,” “zero-config,” “fewer steps”).
- **Leg entries.** For each sink touched: sign, magnitude estimate in that sink’s proxies, and confidence. A record with one leg (only a debit from code) is not a valid transaction — it is the *claim* of reduction and is flagged as such for reconciliation rather than accepted.
- **Receiving parties.** Whose capacity-adjusted cost rises: which user class, which operator role, which maintainer team. Transduction flag set when the receiving party has lower capacity than the sending party.
- **Consent.** Whether the receiving parties agreed, were informed, or neither. The three states — *consensual*, *disclosed*, *externality* — are the accounting’s moral column.
- **Reversibility and deferral terms.** If cost is time-shifted (the deferral sink), the record states the interest terms: expected size at maturity, trigger conditions.

The record format is deliberately boring — a schema, a file, a diff — because the discipline’s power is in the reconciliation, not the capture ceremony.

5.2 The proxies per sink

Each sink’s unit is a field-measurable proxy set, chosen to be already-collected or trivially collectible:

- **Code sink:** static complexity delta on changed surfaces; dependency count delta; configuration-flag count delta (the single most honest proxy for relocated decision complexity — the flag is where a “removed” decision goes to live); migration-surface size.
- **Interaction sink:** added/removed steps on the primary task; new user-held state; new error-recovery obligations; post-change time-on-task delta; accessibility of the moved burden (which percentile of user capacity).
- **Operation sink:** runbook delta; alert-rule delta; toil-hours delta; incident-class changes.
- **Documentation sink:** corpus delta in tasks documented; read-time delta; forum-folklore creation rate (the community’s undocumented absorption — measurable as question volume on the changed surface).
- **Deferral sink:** debt items with size and trigger; workaround birth rate; deprecated-path census.

Proxies are proxies. The ledger does not claim to measure complexity; it claims to *account* the costs complexity generates in units organizations already trust. This is the same epistemic move as GDP, and it is the move that makes the whole thing buildable now.

5.3 Budgets and reconciliation

The **complexity budget** is set at release cadence per surface: an allowed net relocation per sink, with the strong-form prediction (§2.2) implying the rule of thumb — *relocations toward the interaction and deferral sinks should be priced with a multiplier*, because that is where the transduction penalty and the interest live. **Reconciliation** is the release review’s new final act: sum the records; a claimed reduction with no receiving leg triggers a verification task (the exceptional headline event); a net relocation beyond budget triggers the same conversation a cost overrun does; repeated externalities on the same receiving party surface as a governance signal, the way Sarbanes-Oxley made repeated unrecorded transactions a red flag rather than a style.

5.4 The consent principle

The instrument’s normative core, stated as policy rather than law: relocating capacity-adjusted cost to a party without disclosure is an **externality**; with disclosure, a **price**; with agreement, a **trade**. The ledger’s job is to make the third state available and the first state visible. The software-market precedents for each state exist: the “zero-config” product that documents its failure modes prices the relocation honestly; the silent settings removal that surfaces as forum folklore does not. This is deliberately the environmental-impact-assessment pattern: not a prohibition on relocation (some relocations are *correct* — moving complexity from users to experts is usually progress, and the transduction flag’s arrow is the theory’s own statement of which direction progress usually flows) but a disclosure regime that makes the choice legible to the parties who pay.

6 Evaluation design

The theory is testable with data that already exists, which is the cheapest possible form of research:

1. **Retrospective A/B on ticket migration.** For a sample of major consumer-software “simplification” releases (UI redesigns, settings removals, zero-config launches), align support-forum and ticket volume time-series before/after, topic-classified. The relocation theory predicts ticket-migration signatures — volume moving from the removed surface to new failure classes — with timing at release. Reduction theory predicts net decline. The two theories disagree in the data; count the wins.
2. **The waterbed in flag space.** For compiler and build-tool histories (the cleanest public archives), test the relationship between abstraction-simplifying milestones and flag-count trajectory. The folklore predicts flag-space growth as the reservoir for relocated decisions; the measurement is a decade-scale correlation, publicly auditable.
3. **Documentation-volume compensation.** For API redesigns in open-source projects, test whether claimed simplification is followed by documentation and tutorial-volume growth on the new surface (the documentation sink absorbing what the interface shed).
4. **Transduction-penalty asymmetry.** For relocations classifiable by receiving-party capacity (expert→expert vs. expert→civilian), test whether post-change total cost inflates for civilian receivers — the strong form’s signature. Support-ticket labor-hour cost is the operational measure.
5. **Ledger-instrumentation field trial.** One engineering organization runs the record format for two quarters on one product surface; the deliverable is the reconciliation’s effect on design review conversations — does the question “which sink receives this?” change decisions, and

how often does a claimed reduction survive reconciliation. $n=1$ by construction; the program grades it accordingly.

Studies 1–4 are falsification opportunities: if interface-level “simplifications” routinely show *net total-cost reduction* across the joined sinks, the conservation theory is wrong, and the finding — that mass-simplification at product cadence is real — would be more surprising and more useful than the theory. Study 5 measures the instrument’s behavioral effect, not the theory’s truth.

7 What this is not

This is not a *complexity metric* proposal. Cyclomatic complexity, cognitive dimensions, and the maintainability-index family measure artifacts; the ledger accounts transactions between parties. The two are related the way thermometry is related to bookkeeping. This is not a *methodology religion* — the ledger is deliberately agnostic about which design method you use; it only prices what any method moves. This is not a *proposal to never simplify*. Reduction of accidental complexity is the industry’s genuine progress; the ledger exists to distinguish that rare event from its frequent impostor, and to celebrate the former with receipts. This is not a *regulation proposal* — though the disclosure regime would be the natural reference of any future consumer-software disclosure rule, the specification works entirely inside a single engineering organization, today, with files it already owns.

8 Objections, confronted

“Complexity is not a single quantity, so conservation is a category error.” Correct about ontology, irrelevant to accounting. Energy was operationalized by calorimetry before anyone agreed what it *was*; GDP is measured in proxies every day and steers nations. The ledger never sums heterogeneous units into one number — it keeps five accounts in five native proxy sets and requires every claim of reduction to name its receiving leg. Conservation here is a *discipline about transactions*, not a scalar conservation law; the theory’s falsifiable content (§2.2, §6) is stated over the measured accounts.

“The law is unfalsifiable — you can always find complexity somewhere.” The weak form is falsified by any sustained sequence of interface redesigns whose five joined sinks show net total-cost decline; study 1’s design exists precisely because that outcome is measurable and would refute the claim. The theory also forbids its own abuse: it predicts *rare* reduction, quantifiable as the rate at which claimed reductions survive reconciliation — a rate the ledger measures rather than assumes. If that rate turns out to be 90%, Tesler was wrong and the paper will say so with pleasure.

“User-freedom measures will be gamed; the numbers will be massaged.” Some will be — the same is true of financial accounting, whose answer was reconciliation discipline and external audit, not abandoning double-entry. The ledger’s first customer is the honest internal question (“what are we actually doing to our users?”), which does not survive being lied to, and its second customer is any external auditor for whom the records, once they exist, are diffable.

“This adds process to design, which is the last thing design needs.” The ceremony is one record per interface-altering decision and one reconciliation per release — the same order as the ADR practice that strong teams already run. The design *conversation* the ledger replaces (“trust me, it’s simpler”) is not free either; it is paid in the failure classes of §1.

“Reduced-complexity products obviously exist — the smartphone is simpler than the mainframe terminal.” The smartphone is the strongest objection and the theory’s best exhibit. The mainframe’s operating complexity was relocated, not destroyed: out of the end user’s terminal and into the platform, the operator, and a hundred-million-line software stack —

a massive, *consensual*, *expert-ward* relocation, exactly the direction the transduction flag marks as progress. The theory does not say relocations are bad; it says they should be legible. The smartphone story told through the ledger is a story of one of the largest complexity trades in history, priced honestly enough to be repeatable.

9 Limitations

The proxy accounts are coarse and boundary-sensitive: sink definitions are a design choice, and adversarial boundary-drawing can hide transactions the way creative accounting hides costs — the reconciliation’s aggregate view mitigates but cannot eliminate this. Attribution is genuinely hard: post-release ticket migration is confounded by everything else that shipped; the retrospective studies lean on release-timing alignment and class-level topic matching, which narrows but does not close the causal window. The deferral sink’s interest terms are estimates about the future and will frequently be wrong. The strong form’s transduction multiplier will initially be folklore with a number attached — calibrating it is a research program in itself, and the ledger should ship with the multiplier marked as a tunable prior, not a constant. $n=1$ organizational field trials do not generalize. And the theory’s scope is deliberately bounded: it governs interface transactions, not algorithmic discovery — a new algorithm that genuinely reduces essential-complexity-equivalents (quicksort over bubble sort; the FFT) is outside the accounting’s reach, and pretending otherwise would overclaim the instrument.

10 Conclusion

Fifty years ago Tesler wrote down the law and everyone agreed with it in principle and ignored it in practice, because agreeing costs nothing and accounting costs a schema. The schema is this paper’s contribution: five sinks with field-measurable proxies, a double-entry transaction that refuses one-legged claims of reduction, budgets and reconciliation at release cadence, a consent column that distinguishes trades from externalities, and falsifiable predictions whose disproof is as valuable as their proof. The engineering organizations that adopt the discipline get the one artifact the industry has never had: an honest answer, with receipts, to “what did we simplify, and who is paying for it now?” The answer will sometimes be unflattering. That is the instrument working. The law was never that complexity is destiny; the law is that it is *conserved*, *movable*, and *billable* — and the billables are overdue.

References

1. Tesler, L. — the law of conservation of complexity, formulated at Xerox PARC in the 1970s; widely known as Tesler’s Law; his gloss: the inherent complexity “must be dealt with by the user or the designer.” [Design-culture record; high confidence on the attribution and formulation; verification queued for the primary note.]
2. Brooks, F. P. (1986). “No Silver Bullet — Essence and Accident in Software Engineering.” *Information Processing ’86* (IFIP). [Peer-reviewed/proceedings; high confidence.]
3. Ashby, W. R. (1956). *An Introduction to Cybernetics* — the law of requisite variety. [Book; high confidence.]
4. Spolsky, J. (2002). “The Law of Leaky Abstractions.” [Engineering essay; high confidence.]
5. Hyrum’s Law — “users will depend on any observable behavior of your system, documented or not”; named for Hyrum Wright. [Engineering-culture record; high confidence.]
6. Cunningham, W. (1992). Technical debt metaphor, OOPSLA experience report. [Conference record; high confidence.]

7. McCabe, T. (1976). “A Complexity Measure.” *IEEE Transactions on Software Engineering*. [Peer-reviewed; high confidence.]
8. Green, T. R. G. (1989); Green & Petre (1996). Cognitive dimensions of notations. [Research line; high confidence.]
9. Card, S., Moran, T., Newell, A. (1980). The keystroke-level model. *Communications of the ACM*. [Peer-reviewed; high confidence.]
10. The “waterbed effect” in compiler and systems folklore — suppressing complexity in one stage resurfaces it in another. [Folk-lore record; PARTIAL confidence on a canonical citation; verification queued.]
11. Google SRE — toil accounting and alert-budget practice. [Book/practice record; high confidence.]
12. Panko, R. — spreadsheet-error research program (also load-bearing in P-015 of this program). [Research line; high confidence on the program’s existence and magnitudes.]
13. Double-entry bookkeeping and the reconciliation/audit pattern (Pacioli, 1494, for the classic codification). [Historical record; high confidence.]
14. Environmental impact assessment — disclosure regimes for diffuse externalities. [Regulatory pattern; high confidence.]
15. p-rick P-004, *Degradation Contracts* — the pressure-side behavior ledger; the complexity ledger is its design-side counterpart, and the two ledgers share the “queryable record of an invisible transaction” pattern. [Program cross-reference.]
16. p-rick P-006, *The Attention Scheduler* — the interaction sink’s largest single cost (interruption) has a scheduled instrument there; the ledger prices the relocations that fill this sink. [Program cross-reference.]

Working-paper note: references marked “verification queued” are recorded from domain knowledge at high confidence and await the program’s live citation-verification pass (see agents.md); the verification ledger is maintained with the paper sources.